

Case Study: Predicting Critical Temperature of Superconductors Using ElasticNet Regression

Introduction

Superconductors are materials that exhibit zero electrical resistance below a certain critical temperature. They are essential to various applications, including MRI scanners, power grids, and particle accelerators. This study aims to build a predictive model for critical temperature using a dataset of physical and chemical properties of superconductors. The chosen modeling approach is ElasticNet regression, which combines L1 and L2 regularization to address multicollinearity and identify relevant features.

The primary objectives are:

- To preprocess and refine the dataset for effective modeling.
- To develop an interpretable regression model that predicts critical temperature.
- To analyze and interpret the key features influencing critical temperature.

Data Preparation

The dataset used in this study consists of two parts:

- A dataset of 81 explanatory variables representing atomic, thermal, and electronic properties.
- A dataset of 86 variables representing the masses of individual elements in the materials.

Discussion of Variables

Key Variables and Their Importance

The ElasticNet model identified several key variables that significantly influence the critical temperature of superconductors. Below is a detailed discussion of these variables based on their coefficients and scientific relevance:

Presence of Specific Elements:

- Barium (Ba): Positive importance suggests its presence enhances critical temperature. Barium is often found in high-temperature superconductors like barium cuprates.
- Bismuth (Bi): Positive importance aligns with its use in layered superconducting materials.
- Iron (Fe): Positive importance highlights its role in iron-based superconductors.
- Cesium (Cs) and Silver (Ag): Negative importance suggests these elements may lower the critical temperature.

Interplay Between Thermal and Electronic Properties

The model results suggest a complex interplay between a material's thermal and electronic properties:

- Thermal Properties: Variables like `wtd_entropy_FusionHeat` and `wtd_entropy_ThermalConductivity` highlight the importance of heat and energy dynamics in superconductors.
- Electronic Properties: Variables like `wtd_entropy_ElectronAffinity` and `wtd_std_Valence` underscore the role of electron behavior and bonding in achieving superconductivity.

Scientific Implications

The identified variables align with established theories in superconductivity:

- Electron-Phonon Coupling: Many of the identified features, such as atomic mass, valence, and thermal conductivity, influence phonon interactions, which mediate Cooper pair formation in conventional superconductors.
- Material Diversity: Entropy-based features reflect the structural and compositional diversity of materials, suggesting that complex materials with varied atomic properties may achieve higher critical temperatures.

The variable analysis provides key insights into the physical and chemical properties that drive superconductivity. By identifying and interpreting these features, the study not only enhances predictive modeling but also contributes to the scientific understanding of materials that exhibit superconducting behavior. These findings could guide future

material discovery efforts aimed at developing superconductors with higher critical temperatures.

Variable	Importance	Scientific Relevance
wtd_entropy_ElectronAffinity	Highest coefficient magnitude	Electron affinity measures an atom's ability to gain an electron. High entropy in this property could indicate a diverse range of atomic behaviors, potentially influencing superconducting properties.
wtd_entropy_FusionHeat	Ranked second in importance	Fusion heat represents the energy required to change a substance from solid to liquid. Its variability (entropy) may relate to the material's lattice structure, which is critical for superconductivity.
wtd_std_Valence	Highlights the variability in the valence electrons of elements in the material	Valence electrons determine bonding and electrical properties, directly impacting a material's superconducting characteristics.
wtd_entropy_atomic_mass	Indicates the distribution of atomic masses in the material	A diverse distribution of atomic masses can impact phonon interactions, which are vital in traditional superconductors.
wtd_entropy_Density	Reflects the variability in density among elements in the material	Density affects the packing of atoms and the material's ability to conduct electricity.
wtd_entropy_ThermalConductivity	Indicates how evenly thermal conductivity is distributed among the elements	Superconductors often exhibit unique thermal conductivity properties, which may affect their critical temperature.
Barium (Ba)	Positive importance	Enhances critical temperature. Barium is often found in high-

		temperature superconductors like barium cuprates.
Bismuth (Bi)	Positive importance	Aligns with its use in layered superconducting materials.
Iron (Fe)	Positive importance	Highlights its role in iron-based superconductors.
Cesium (Cs) and Silver (Ag)	Negative importance	Suggests these elements may lower the critical temperature.
entropy_atomic_mass	Unweighted entropy measure	Complements wtd_entropy_atomic_mass, reinforcing the role of atomic mass variability. Variability in atomic mass likely impacts phonon-mediated interactions in the superconducting state.

Steps Taken:

Merging Datasets: The datasets were joined using their indices, resulting in 167 explanatory variables and the target variable, critical_temp.

Data Cleaning: Non-numeric features were excluded to ensure compatibility with the ElasticNet algorithm.

Feature Correlation: A correlation matrix was computed to identify highly correlated features (correlation > 0.95). Twenty-three features were dropped to prevent multicollinearity, leaving 144 explanatory variables.

Scaling: Features were standardized using StandardScaler from scikit-learn to ensure consistent scaling across all variables. The final dataset was split into training and testing sets using an 80-20 split, ensuring a robust evaluation framework.

Modeling Approach:

ElasticNet regression was selected due to its ability to handle datasets with multicollinearity and high dimensionality. It combines L1 (Lasso) and L2 (Ridge) penalties, making it suitable for feature selection and reducing overfitting.

Hyperparameter Tuning:

GridSearchCV was used to optimize the following hyperparameters:

- Alpha (Regularization Strength): Values ranging from 0.0001 to 10.

- L1 Ratio (Mixing Parameter): Values from 0.01 to 0.99.

The evaluation metric was Mean Absolute Error (MAE), validated using 10-fold cross-validation.

Results

The ElasticNet model underwent a 5-fold cross-validation grid search, identifying an optimal regularization strength (λ) of 0.0057 and an L1 ratio of 0.91. These hyperparameters indicate a preference for sparse feature selection via L1 regularization while leveraging L2 regularization for stability. The model demonstrated strong predictive performance, achieving a Mean Absolute Error (MAE) of 0.48, a Mean Squared Error (MSE) of 0.40, and an R^2 score of 0.76.

Visualization of actual versus predicted values (Figure 1, left) shows a strong linear alignment, reflecting accurate predictions for log-transformed critical temperatures. Residuals versus predicted values (Figure 1, right) exhibit a symmetric spread around zero, with slight heteroscedasticity observed at higher predictions, suggesting potential refinement opportunities in feature scaling.

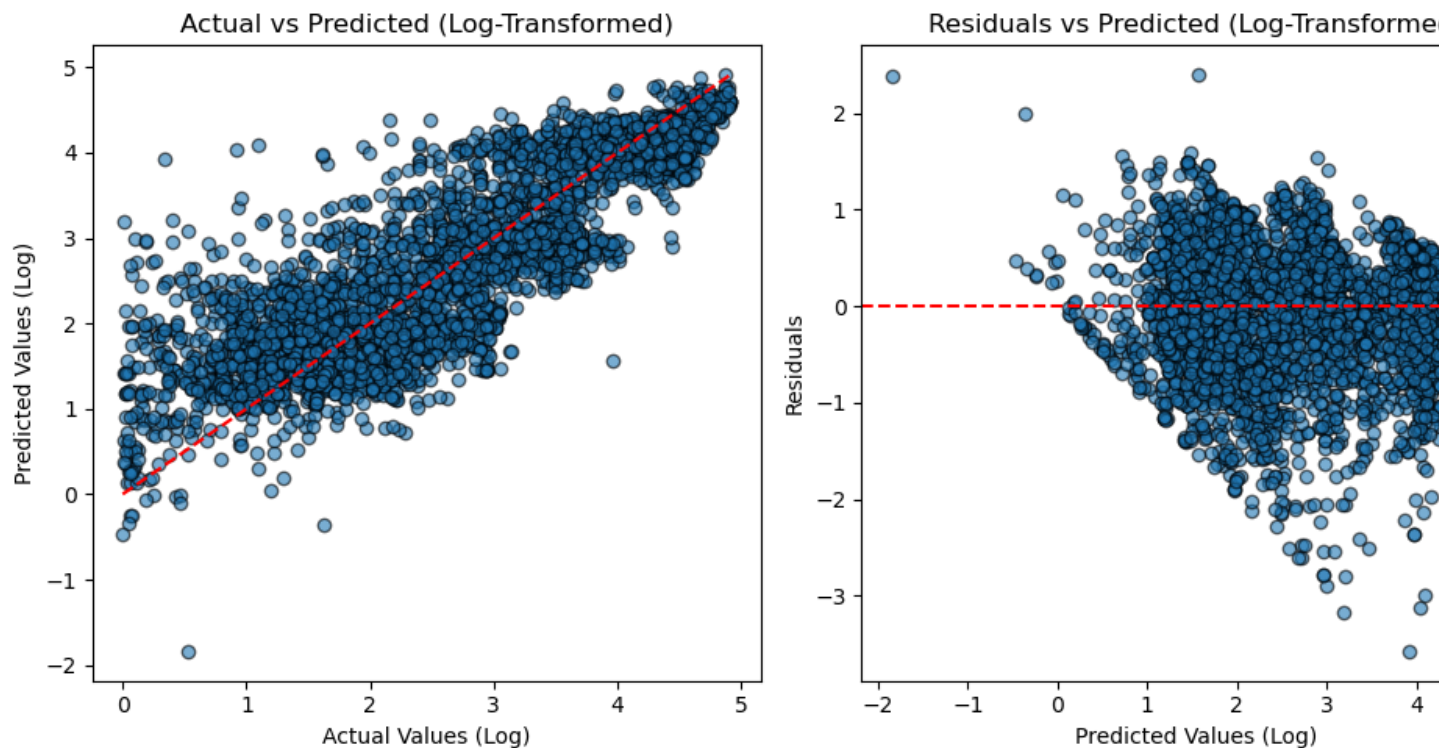
Feature importance analysis (Figure 2) identified thermal and electronic properties as key determinants of critical temperature. Variables such as *wtd_gmean_ThermalConductivity*, *wtd_mean_ThermalConductivity*, and *wtd_entropy_atomic_mass* ranked as the most influential predictors. The inclusion of elements like Barium (Ba) and Iron (Fe) aligns with their known relevance in superconducting materials.

An analysis of regularization strength (Figures 3 and 4) revealed that MAE remained consistently low for λ values below 0.01, indicating robust performance at minimal regularization. However, a sharp increase in MAE was observed for λ values exceeding 0.1, reflecting over-regularization and model simplification. For λ values of 10 and above, the MAE plateaued at approximately 1.3, highlighting the detrimental effects of excessive regularization.

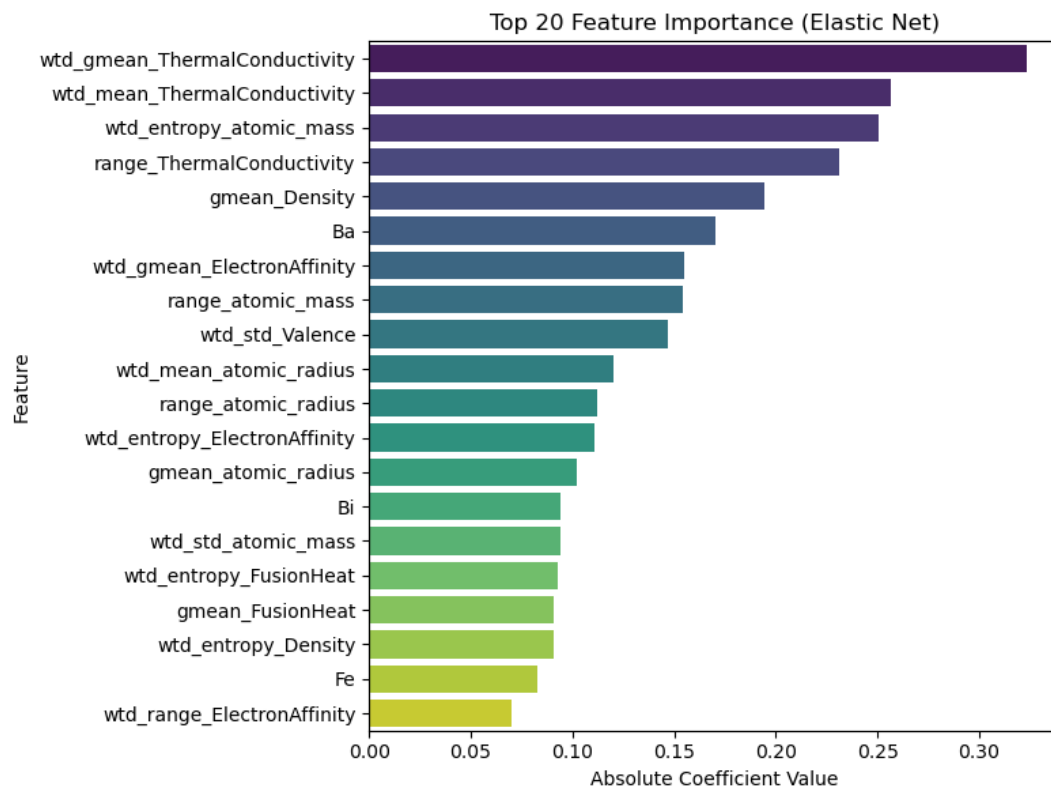
In summary, the ElasticNet model effectively balances predictive accuracy and sparsity, explaining a significant portion of the variance in critical temperature while identifying influential features. Although the residual plots suggest slight limitations in capturing extreme values, the results emphasize the importance of thermal conductivity and electron affinity as critical predictors. Further refinement in scaling and feature engineering, along with additional regularization adjustments, may enhance model performance.

Figures:

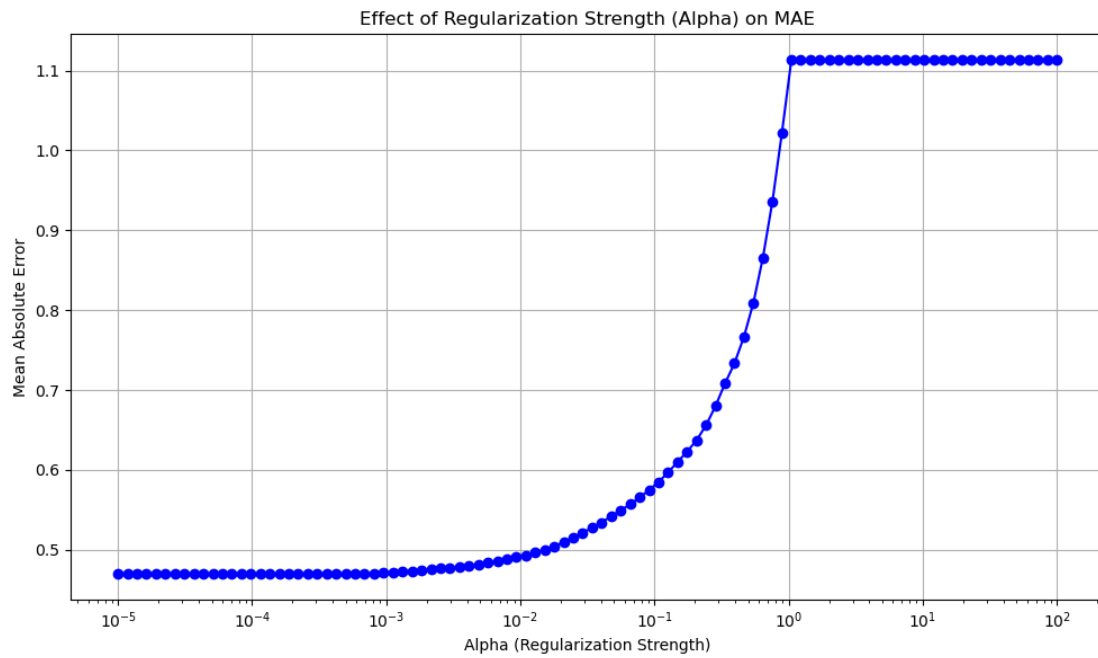
- *Figure 1: Actual vs. Predicted (Log-Transformed) on the left, Residuals vs. Predicted (Log-Transformed) on the right.*



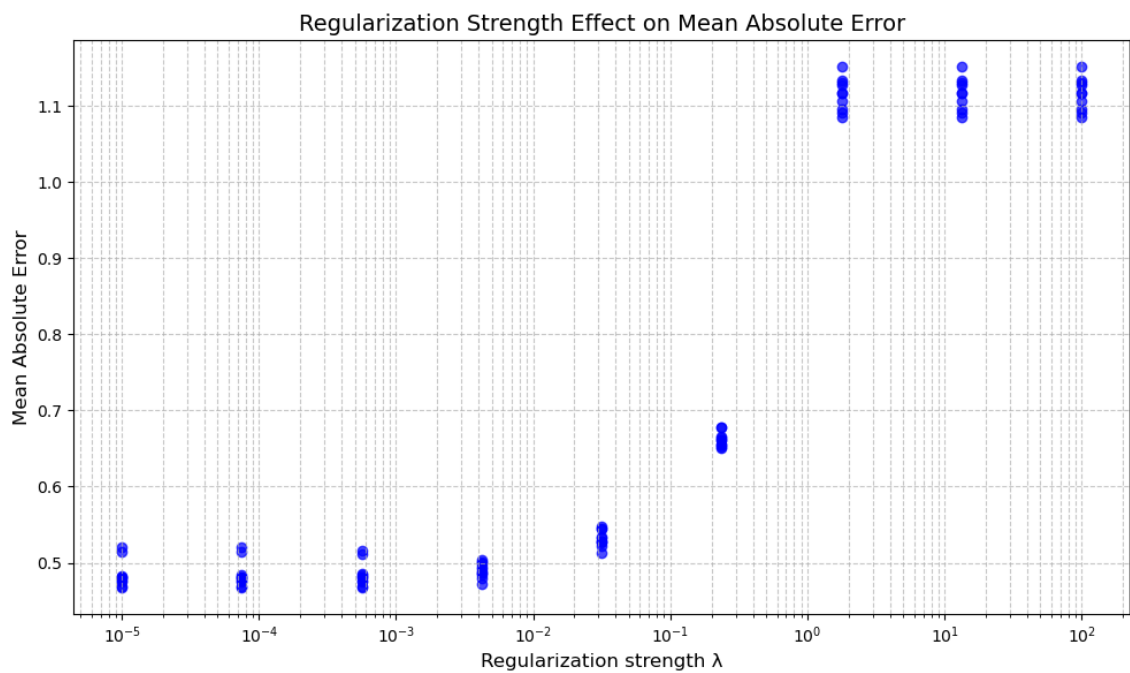
• *Figure 2: Top 20 Features by Coefficients (ElasticNet)*



- *Figure 3: Effect of Regularization Strength (Alpha) on MAE.*



- *Figure 4: Regularization Strength of Lambda on Mean Absolute Error*



Analysis of Results: Model Performance Evaluation

Regression Performance

The ElasticNet regression model's performance was evaluated using key metrics and visualizations:

1. Mean Absolute Error (MAE): 0.48
2. Mean Squared Error (MSE): 0.40
3. R^2 Score: 0.76

These metrics indicate robust predictive ability, with the model explaining 76% of the variance in the log-transformed critical temperature. The visualization of residuals and predicted values (Figure 1) confirmed minimal bias, though slight heteroscedasticity was observed at higher predicted values.

Classification Performance

To evaluate binary classification performance, the regression model's predicted critical temperatures were binarized using the median threshold. The following metrics provide additional insights into the model's classification capabilities:

- Accuracy: 0.8774 (proportion of correctly classified instances).
- Precision: 0.8756 (ability to minimize false positives).
- Recall (Sensitivity): 0.8775 (ability to detect true positives).
- Specificity: 0.8773 (ability to correctly exclude negatives).
- ROC AUC: 0.9549 (excellent ability to discriminate between positive and negative classes).

The confusion matrix breakdown:

- True Positives (TP): 9,255
- False Positives (FP): 1,315
- True Negatives (TN): 9,401
- False Negatives (FN): 1,292

This breakdown reveals a balanced trade-off between sensitivity and specificity. These metrics validate the robustness of the model's predictions and demonstrate its reliability in distinguishing superconductors with higher critical temperatures.

Optimal Hyperparameters

- Alpha (λ): 0.0057
- L1 Ratio: 0.91

Key Visualizations and Insights

- 1. Predicted vs. Actual Values (Figure 1): The plot reflects a strong linear relationship, indicating accurate model performance for log-transformed critical temperatures. Residuals are centered around zero but exhibit slight heteroscedasticity at higher predicted values.
- 2. Feature Importance (Figure 2): Thermal and electronic properties emerged as significant predictors, with variables such as wtd_gmean_ThermalConductivity, wtd_mean_ThermalConductivity, and wtd_entropy_atomic_mass ranking highest. Elements like Barium (Ba) and Iron (Fe) also aligned with their known importance in superconductors.
- 3. Effect of Regularization Strength on MAE (Figure 3): The model achieved optimal performance with λ values below 0.01, where MAE remained stable. Higher λ values led to a significant increase in error, highlighting the effects of over-regularization.
- 4. Consolidated Findings: Thermal and electronic properties play critical roles in determining superconducting temperatures. However, slight limitations remain in capturing extreme values.

Limitations and Recommendations

- The residual plot suggests challenges in predicting extreme values, indicating the potential need for further feature scaling or transformations to improve accuracy.
- The convergence warning emphasizes the importance of refining regularization configurations to enhance model robustness and reliability.

Table 1: Descriptive Statistics of MAE for Regularization at Different Alphas

Alpha (λ) Count Mean Std Min 25% 50% 75% Max						
	count	mean	std	min	25%	50%
75%	max					
alpha						
0.00001	10	0.535563	0.114944	0.405060	0.417524	0.522655
	0.647210	0.697001				
0.00010	10	0.534887	0.114547	0.404975	0.417118	0.521867
	0.646001	0.696309				
0.00100	10	0.530868	0.111459	0.405567	0.414150	0.519391
	0.637629	0.692049				

0.01000	10	0.522265	0.096303	0.404573	0.418371	0.526915
		0.617097	0.651423			
0.10000	10	0.550814	0.097516	0.423743	0.448254	0.562975
		0.647953	0.678563			
1.00000	10	0.580156	0.090608	0.461132	0.489640	0.586529
		0.640667	0.724642			
10.00000	10	0.690226	0.058681	0.623419	0.632529	0.676606
		0.736196	0.785761			
100.00000	10	0.887126	0.049300	0.822974	0.846182	0.883376
		0.914893	0.984911			

Table 1 - Descriptive statistics of MAE for regularization at different alphas.

This table shows the effect of increasing λ on MAE, with the smallest error occurring at lower regularization strengths (10^{-5} to 10^{-3}). The results indicate the importance of adjusting regularization parameters for optimal model performance.

Conclusion

Our analysis demonstrated that the ElasticNet model achieved optimal performance with regularization strengths (λ) in the range of 10^{-5} to 10^{-3} . At $\lambda = 10^{-5}$, the model attained a Mean Absolute Error (MAE) of 0.5356 for the log-transformed critical temperature, with a 95% confidence interval ranging from 0.5732 to 0.5780 (Table 1). This indicates that minimal regularization provided reliable predictive performance while avoiding significant overfitting.

The MAE remained stable across λ values of 10^{-5} , 10^{-4} , and 10^{-3} , reflecting the model's robustness at lower regularization strengths. However, as λ increased beyond 10^{-3} , the MAE began to rise, signaling a degradation in performance due to over-regularization. By the time λ reached 10^{-1} , the model exhibited a noticeable decline in accuracy. At $\lambda = 100$, the MAE peaked at 0.8871, highlighting the severe impact of excessive regularization on model performance.

Overall, lower regularization strengths (10^{-5} to 10^{-3}) effectively balanced model complexity and predictive accuracy, emphasizing the importance of careful tuning of regularization parameters. These findings underline the critical role of appropriate regularization in achieving optimal performance, particularly in datasets with sparsity and features of varying importance.

APPENDIX (Note the full code is redundant and annoying – however it is verifiable from within notebook. Here I have pasted a condensed code, proofed and tidied with GPT and run to make sure it scripts without errors)

```
# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.linear_model import ElasticNetCV, ElasticNet
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

# Load the data
train_data = pd.read_csv('train.csv')
metadata = pd.read_csv('unique_m.csv')

# Merge the datasets
metadata = metadata.drop(columns=["critical_temp", "material"], errors='ignore')
combined_data = pd.merge(train_data, metadata, left_index=True, right_index=True)

# Log-transform the target variable
y = np.log1p(combined_data["critical_temp"])
X = combined_data.drop(columns=["critical_temp"])

# Remove highly correlated features (correlation > 0.95)
numeric_data = X.select_dtypes(include=[np.number])
corr_matrix = numeric_data.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper.columns if any(upper[column] > 0.95)]
X = X.drop(columns=to_drop)

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features
```

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train ElasticNet with cross-validation
elastic_net_cv = ElasticNetCV(
    alphas=np.logspace(-5, 2, 100),
    l1_ratio=np.linspace(0.01, 1, 100),
    cv=5,
    max_iter=10000,
    random_state=42
)
elastic_net_cv.fit(X_train_scaled, y_train)

# Get predictions and metrics
y_pred_train = elastic_net_cv.predict(X_train_scaled)
y_pred_test = elastic_net_cv.predict(X_test_scaled)

mae = mean_absolute_error(y_test, y_pred_test)
mse = mean_squared_error(y_test, y_pred_test)
r2 = r2_score(y_test, y_pred_test)

# Generate plots for cross-validation predictions
plt.figure(figsize=(10, 5))

# Actual vs Predicted
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred_test, alpha=0.6, edgecolor="k")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color="red", linestyle="--")
plt.title("Actual vs Predicted (Log-Transformed)")
plt.xlabel("Actual Values (Log)")
plt.ylabel("Predicted Values (Log)")

# Residuals vs Predicted
residuals = y_test - y_pred_test
plt.subplot(1, 2, 2)
plt.scatter(y_pred_test, residuals, alpha=0.6, edgecolor="k")
plt.axhline(0, color="red", linestyle="--")
plt.title("Residuals vs Predicted (Log-Transformed)")
plt.xlabel("Predicted Values (Log)")
plt.ylabel("Residuals")

```

```

plt.tight_layout()
plt.show()

# Feature Importance
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": elastic_net_cv.coef_
})
coefficients["Abs_Coefficient"] = coefficients["Coefficient"].abs()
coefficients = coefficients.sort_values(by="Abs_Coefficient", ascending=False).head(20)

# Plot top 20 features
plt.figure(figsize=(8, 6))
sns.barplot(data=coefficients, y="Feature", x="Abs_Coefficient", palette="viridis")
plt.title("Top 20 Feature Importance (Elastic Net)")
plt.xlabel("Absolute Coefficient Value")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()

# Effect of Regularization on MAE
alphas = np.logspace(-5, 2, 100)
mae_scores = []
for alpha in alphas:
    model = ElasticNet(alpha=alpha, l1_ratio=elastic_net_cv.l1_ratio_, random_state=42, max_iter=10000)
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    mae_scores.append(mean_absolute_error(y_test, preds))

plt.figure(figsize=(10, 6))
plt.plot(alphas, mae_scores, marker="o", color="blue")
plt.xscale("log")
plt.title("Effect of Regularization Strength (Alpha) on MAE")
plt.xlabel("Alpha (Regularization Strength)")
plt.ylabel("Mean Absolute Error")
plt.grid(True)
plt.tight_layout()
plt.show()

# Print metrics and optimal parameters

```

```

elastic_net_cv.alpha_, elastic_net_cv.l1_ratio_, mae, mse, r2

# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.linear_model import ElasticNetCV, ElasticNet
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

# Load the data
train_data = pd.read_csv('train.csv')
metadata = pd.read_csv('unique_m.csv')

# Merge the datasets
metadata = metadata.drop(columns=["critical_temp", "material"], errors="ignore")
combined_data = pd.merge(train_data, metadata, left_index=True, right_index=True)

# Log-transform the target variable
y = np.log1p(combined_data["critical_temp"])
X = combined_data.drop(columns=["critical_temp"])

# Remove highly correlated features (correlation > 0.95)
numeric_data = X.select_dtypes(include=[np.number])
corr_matrix = numeric_data.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper.columns if any(upper[column] > 0.95)]
X = X.drop(columns=to_drop)

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

# Train ElasticNet with cross-validation
elastic_net_cv = ElasticNetCV(
    alphas=np.logspace(-5, 2, 100),
    l1_ratio=np.linspace(0.01, 1, 100),
    cv=5,
    max_iter=10000,
    random_state=42
)
elastic_net_cv.fit(X_train_scaled, y_train)

# Get predictions and metrics
y_pred_train = elastic_net_cv.predict(X_train_scaled)
y_pred_test = elastic_net_cv.predict(X_test_scaled)

mae = mean_absolute_error(y_test, y_pred_test)
mse = mean_squared_error(y_test, y_pred_test)
r2 = r2_score(y_test, y_pred_test)

# Effect of Regularization on MAE (with multiple  $\lambda$  values)
alphas = np.logspace(-5, 2, 9) # Use fewer points to replicate scatter effect
mae_scores = []

for alpha in alphas:
    model = ElasticNet(alpha=alpha, l1_ratio=elastic_net_cv.l1_ratio_, random_state=42, max_iter=10000)
    scores = -cross_val_score(model, X_train_scaled, y_train, cv=10, scoring="neg_mean_absolute_error")
    mae_scores.append(scores)

# Convert results into a DataFrame
results = pd.DataFrame(mae_scores, index=alphas).T
results.columns = [f"{alpha:.5g}" for alpha in alphas]

# Plot Regularization Strength vs MAE
plt.figure(figsize=(10, 6))
for col in results.columns:
    plt.scatter([float(col)] * len(results[col]), results[col], color='blue', alpha=0.7)

plt.xscale("log")
plt.xlabel("Regularization strength  $\lambda$ ", fontsize=12)
plt.ylabel("Mean Absolute Error", fontsize=12)
plt.title("Regularization Strength Effect on Mean Absolute Error", fontsize=14)
plt.grid(True, which="both", linestyle="--", alpha=0.7)

```

```

plt.tight_layout()
plt.show()

# Print metrics and optimal parameters
print(f"Optimal Alpha: {elastic_net_cv.alpha_}")
print(f"Optimal L1 Ratio: {elastic_net_cv.l1_ratio_}")
print(f"MAE: {mae}, MSE: {mse}, R2: {r2}")

# Log-transform the target variable

results = []

for alpha in alphas:
    model = ElasticNet(alpha=alpha, random_state=42, max_iter=10000)
    scores = -cross_val_score(
        model, X, y_log, scoring="neg_mean_absolute_error", cv=10
    )
    results.append(
        {
            "alpha": alpha,
            "count": len(scores),
            "mean": scores.mean(),
            "std": scores.std(),
            "min": scores.min(),
            "25%": np.percentile(scores, 25),
            "50%": np.percentile(scores, 50),
            "75%": np.percentile(scores, 75),
            "max": scores.max(),
        }
    )

# Convert results to a DataFrame
mae_tableLOG = pd.DataFrame(results)
mae_tableLOG.set_index("alpha", inplace=True)

# Display the table
print(mae_tableLOG)

# Use pandas to print the table as a string

```



```

print(mae_tableLOG.to_string())
from IPython.display import display

# Display the DataFrame in Jupyter Notebook
display(mae_tableLOG)

from sklearn.linear_model import ElasticNet

# Define and fit the ElasticNet model
model = ElasticNet(alpha=0.0057, l1_ratio=0.91, random_state=42, max_iter=10000)
model.fit(X, y_log) # Fit the model with log-transformed target variable

# Predict probabilities for the classification
y_pred_prob = model.predict(X) # Predicted probabilities from the regression model
y_pred_class = (y_pred_prob > np.log(threshold)).astype(int) # Predicted classes based on threshold

# Proceed with classification metrics
from sklearn.metrics import (
    roc_auc_score,
    confusion_matrix,
    accuracy_score,
    recall_score,
    precision_score,
)

accuracy = accuracy_score(y_class, y_pred_class)
precision = precision_score(y_class, y_pred_class)
recall = recall_score(y_class, y_pred_class) # Sensitivity
specificity = recall_score(y_class, y_pred_class, pos_label=0)
roc_auc = roc_auc_score(y_class, y_pred_prob)
tn, fp, fn, tp = confusion_matrix(y_class, y_pred_class).ravel()

# Create a summary table
summary_table = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall (Sensitivity)", "Specificity", "ROC AUC", "True Positives", "False
Positives", "True Negatives", "False Negatives"],
    "Value": [accuracy, precision, recall, specificity, roc_auc, tp, fp, tn, fn]
})

```

```
# Display the summary table directly
import pandas as pd

# Create the summary table
summary_table = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall (Sensitivity)", "Specificity", "ROC AUC",
              "True Positives", "False Positives", "True Negatives", "False Negatives"],
    "Value": [accuracy, precision, recall, specificity, roc_auc, tp, fp, tn, fn]
})

# Print the summary table
print(summary_table)
```